

Contributing to Athena

Table of Contents

- [Contributing to Athena](#)

- [Development Setup](#) - [Prerequisites](#) - [Getting Started](#) - [Code Style](#) - [TypeScript Conventions](#) - [File Organization](#) - [Code Patterns](#) - [Making Changes](#) - [Branch Naming](#) - [Commit Messages](#) - [Development Workflow](#) - [Testing](#) - [Running Tests](#) - [What to Test](#) - [Testing Checklist](#) - [Pull Request Process](#) - [Before Submitting](#) - [PR Template](#) - [Review Expectations](#) - [After Approval](#) - [Issue Reporting](#) - [Bug Reports](#) - [Feature Requests](#) - [Questions and Discussions](#) - [Code of Conduct](#) - [Our Standards](#) - [Unacceptable Behavior](#) - [Enforcement](#) - [Questions?](#) - [Related Documentation](#)

Welcome! 🙌 Thank you for your interest in contributing to Athena. This project aims to help developers maintain comprehensive documentation through automated analysis and AI-powered generation. We welcome contributions from developers of all experience levels.

Development Setup

Prerequisites

- Node.js (v16 or higher recommended)
- npm or yarn package manager
- Git for version control

Getting Started

1. Fork and clone the repository

```
git clone https://github.com/YOUR_USERNAME/athena.git
cd athena
```

2. Install dependencies

```
npm install
```

3. Build the project

```
npm run build
```

4. Run in development mode

```
npm run dev
```

5. Test the CLI

```
npm start -- --help
```

The project is written in TypeScript and uses `tsx` for development execution and `tsc` for production builds.

Code Style

TypeScript Conventions

- Use TypeScript for all source files (`.ts` extension)
- Follow `async/await` patterns for asynchronous operations
- Use explicit function signatures with types
- Prefer named exports over default exports
- Use descriptive variable and function names

File Organization

- `src/` — main source code
- `generators/` — documentation generation logic - `scanners/` — project analysis and scanning - `diagrams/` — diagram generation (Mermaid) - `screenshots/` — UI screenshot capabilities - `utils/` — shared utility functions - `changeLog/` — git history parsing
- `dist/` — compiled JavaScript output (generated)

Code Patterns

- Async functions should use descriptive names (e.g., `async scanProject()`, `async loadConfig()`)
- Utility functions should be pure and testable (e.g., `slugify()`, `parseMarkdownSections()`)
- Use helper functions for formatting and summarization (e.g., `summarizeText()`, `formatScreenshotEmbed()`)
- Classes should have clear responsibilities (e.g., `DocGenerator`, `ClaudeClient`, `DiagramGenerator`)

Making Changes

Branch Naming

Use descriptive branch names following these patterns:

- `feature/add-new-scanner` — for new features
- `fix/correct-markdown-parsing` — for bug fixes
- `docs/update-readme` — for documentation updates
- `refactor/improve-generator` — for code improvements
- `test/add-scanner-tests` — for test additions

Commit Messages

Write clear, concise commit messages:

- Use present tense ("Add feature" not "Added feature")
- Use imperative mood ("Fix bug" not "Fixes bug")
- First line should be 50 characters or less
- Reference issues when applicable (e.g., "Fix #123: Resolve config loading")

Examples:

```
Add support for Python project scanning
Fix markdown section parsing for nested headings

Improve error messages in CLI output

Update CONTRIBUTING.md with TypeScript guidelines
```

Development Workflow

1. Create a new branch from `main`
2. Make your changes, following code style guidelines
3. Build the project to check for TypeScript errors: `npm run build`
4. Test your changes manually with `npm run dev`
5. Commit your changes with clear messages
6. Push to your fork and create a pull request

Testing

Running Tests

Currently, the project uses manual testing workflows. To test your changes:

1. Build the project

```
npm run build
```

2. Test CLI functionality

```
npm start -- [command] [options]
```

3. Test in development mode

```
npm run dev -- [command] [options]
```

What to Test

When making changes, ensure you test:

- **Scanner functionality** — Test `scanProject()` with different project types
- **Config loading** — Verify `loadConfig()` handles various configurations
- **Document generation** — Test `DocGenerator.generate()` output quality
- **Markdown utilities** — Verify `parseMarkdownSections()`, `generateTableOfContents()` work correctly
- **CLI commands** — Test all CLI options and flags
- **Error handling** — Ensure graceful failures with helpful messages
- **Edge cases** — Test with missing files, invalid configs, empty projects

Testing Checklist

Before submitting a PR, verify:

- ☐ Project builds without TypeScript errors
- ☐ CLI runs without crashes
- ☐ New features work as documented
- ☐ Existing functionality isn't broken
- ☐ Error messages are helpful
- ☐ Code handles edge cases gracefully

Pull Request Process

Before Submitting

1. Ensure your code builds: `npm run build`
2. Update documentation if you changed functionality
3. Add comments for complex logic
4. Verify all TypeScript types are correct

PR Template

When creating a pull request, include:

Description

- What changes does this PR introduce?
- Why are these changes needed?

Type of Change

- ☐ Bug fix
- ☐ New feature
- ☐ Breaking change
- ☐ Documentation update

Testing

- How were these changes tested?
- What edge cases were considered?

Related Issues

- Closes #[issue number]
- Related to #[issue number]

Review Expectations

- Maintainers will review PRs as time permits
- You may be asked to make changes or clarifications
- Be responsive to feedback and questions
- Keep discussions respectful and constructive
- Reviews focus on code quality, maintainability, and project fit

After Approval

Once approved, maintainers will merge your PR. Thank you for your contribution!

Issue Reporting

Bug Reports

When filing a bug report, include:

1. **Description** — Clear description of the bug
2. **Steps to Reproduce** — Exact steps to trigger the issue
3. **Expected Behavior** — What should happen
4. **Actual Behavior** — What actually happens
5. **Environment** — OS, Node version, project type
6. **Logs/Screenshots** — Any relevant error messages

Example:

Bug: Config file not loading in subdirectories

Steps:

1. Create config in src/.athena.json
2. Run `athena scan` from project root
3. Config is not detected

Expected: Config should be found and loaded

Actual: Uses default config instead

Environment: macOS 14, Node 18.16.0

Feature Requests

For feature requests, describe:

1. **Problem** — What problem would this solve?
2. **Solution** — Your proposed solution
3. **Alternatives** — Other approaches you considered
4. **Use Case** — How you would use this feature

Questions and Discussions

For questions or general discussions:

- Check existing issues first
- Use clear, descriptive titles
- Provide context about what you're trying to achieve
- Be patient and respectful

Code of Conduct

Our Standards

We are committed to providing a welcoming and inclusive environment. We expect all contributors to:

- Use welcoming and inclusive language
- Respect differing viewpoints and experiences
- Accept constructive criticism gracefully
- Focus on what's best for the project and community
- Show empathy toward other community members

Unacceptable Behavior

Examples of unacceptable behavior include:

- Harassment, trolling, or insulting comments
- Personal or political attacks
- Publishing others' private information
- Any conduct inappropriate in a professional setting

Enforcement

Project maintainers have the right to remove, edit, or reject contributions that don't align with this Code of Conduct. Contributors who violate these standards may be temporarily or permanently banned from the project.

Questions?

If you have questions about contributing, feel free to:

- Open an issue for discussion
- Review existing issues and PRs for examples
- Reach out to maintainers

Thank you for contributing to Athena! 🚀

Related Documentation

- [Readme](#)

- [Architecture](#)
- [Api](#)
- [Deployment](#)

Generated by Athena — © 2026 TheForge, LLC